

A. Kyoya and Colored Balls

time limit per test: 2 seconds
memory limit per test: 256 megabytes

Kyoya Ootori has a bag with n colored balls that are colored with k different colors. The colors are labeled from 1 to k . Balls of the same color are indistinguishable. He draws balls from the bag one by one until the bag is empty. He noticed that he drew the last ball of color i before drawing the last ball of color $i + 1$ for all i from 1 to $k - 1$. Now he wonders how many different ways this can happen.

Input

The first line of input will have one integer k ($1 \leq k \leq 1000$) the number of colors.

Then, k lines will follow. The i -th line will contain c_i , the number of balls of the i -th color ($1 \leq c_i \leq 1000$).

The total number of balls doesn't exceed 1000.

Output

A single integer, the number of ways that Kyoya can draw the balls from the bag as described in the statement, modulo 1 000 000 007.

Examples

input	Copy
3 2 2 1	
output	Copy
3	

input	Copy
4 1 2 3 4	
output	Copy
1680	

Note

In the first sample, we have 2 balls of color 1, 2 balls of color 2, and 1 ball of color 3. The three ways for Kyoya are:

1 2 1 2 3
1 1 2 2 3
2 1 1 2 3

Problem restated in simple words

- There are k colors: 1, 2, ..., k
- Color i has c_i identical balls
- Total balls ≤ 1000
- We draw all balls one by one (a sequence of length $\sum c_i$)

Constraint (very important)

The last ball of color i is drawn before the last ball of color $i+1$,
for all $i = 1 \dots k-1$

We must count how many draw sequences satisfy this.

Understand what the constraint really means

Let's define:

$last(i)$ = position of the last occurrence of color i in the sequence

The condition is:

$last(1) < last(2) < last(3) < \dots < last(k)$

This does NOT say:

- all balls of color 1 come before color 2
- or that the sequence is sorted

Balls can interleave freely — only the last occurrences are ordered.

Fix the "last balls"

For each color i , mark one special ball:

the last ball of color i

Because of the constraint, these special balls must appear in this order:

$L_1 \ L_2 \ L_3 \ \dots \ L_k$

This order is fixed — we don't count it, we assume it.

Now, all remaining balls are non-last balls.

What remains to be placed?

For color i :

- Total balls = c_i
- One is the "last" ball
- Remaining = $c_i - 1$ identical balls

These $c_i - 1$ balls must be placed before L_i .

Build the sequence step by step (stars and bars view)

We process colors in order:

$1 \rightarrow 2 \rightarrow 3 \rightarrow \dots \rightarrow k$

Assume we are placing color i .

At this moment:

Already placed:

$S = c_1 + c_2 + \dots + c_{i-1}$

balls (including last balls of previous colors).

Now we want to place:

$c_i - 1$

identical balls of color i , before L_i .

Visual stars and bars interpretation (core idea)

Think of the existing S balls as stars:

$* \ * \ * \ * \ \dots \ *$ (S stars)

Between these stars, and at the ends, there are gaps where we can insert balls:

$| \ * \ | \ * \ | \ * \ | \ \dots \ | \ * \ |$

Number of gaps:

$S + 1$

We want to distribute $c_i - 1$ identical balls into these $S + 1$ gaps.

This is exactly a stars and bars problem:

$y_1 + y_2 + \dots + y_{(S+1)} = c_i - 1$
 $y_i \geq 0$

Number of ways to do this is:

$$\binom{(S+1) + (c_i - 1) - 1}{(c_i - 1)} = \binom{S + c_i - 1}{c_i - 1}$$

Final Formula

$$\text{Answer} = \prod_{i=1}^k \binom{(c_1 + c_2 + \dots + c_{i-1}) + c_i - 1}{c_i - 1}$$

Code

```
#include <bits/stdc++.h>
using namespace std;

static const long long MOD = 1000000007;
static const int MAXN = 2000;

long long fact[MAXN + 1], invfact[MAXN + 1];

long long modpow(long long a, long long b) {
    long long res = 1;
    while (b) {
        if (b & 1) res = res * a % MOD;
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

void precompute() {
    fact[0] = 1;
    for (int i = 1; i <= MAXN; i++)
        fact[i] = fact[i - 1] * i % MOD;

    invfact[MAXN] = modpow(fact[MAXN], MOD - 2);
    for (int i = MAXN; i > 0; i--)
        invfact[i - 1] = invfact[i] * i % MOD;
}

long long nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    precompute();

    int k;
    cin >> k;

    long long ans = 1;
    int S = 0;

    for (int i = 1; i <= k; i++) {
        int ci;
        cin >> ci;

        ans = ans * nCr(S + ci - 1, ci - 1) % MOD;
        S += ci;
    }

    cout << ans << "\n";
    return 0;
}
```